Specification - Rob INTF



Bioyond Robotics

Version: 1.1

All rights reserved © Shanghai Bioyond Technology Co., Ltd

1. Introduction

This document describes the communication protocol between Bioyond robot controller and a host controller, named Rob INTF, an abbreviation of "robot interface". With this interface, the host controller can perform simple operations on the robot without the need for a robot TPU, including switching control modes, getting the robot status, clearing errors, enable/disable, and making the robot execute the pre-prepared programs. Any host controller with this interface, such as a central controller with scheduling capabilities, can treat the Bioyond robot as a device in the entire automation system, so as to control the robot and other devices to work together to complete specific tasks.

All rights reserved © Shanghai Bioyond Technology Co., Ltd

2. Overview

The Rob INTF overview structure is based on the TCP socket, shown in Fig. 1-1. In this system , the robot controller acts as the server, while the host controller is the client. It should be noted that at present, the robot controller as a server only supports the connection with one client at a time, and does not support multiple clients to connect and communicate with it at the same time.

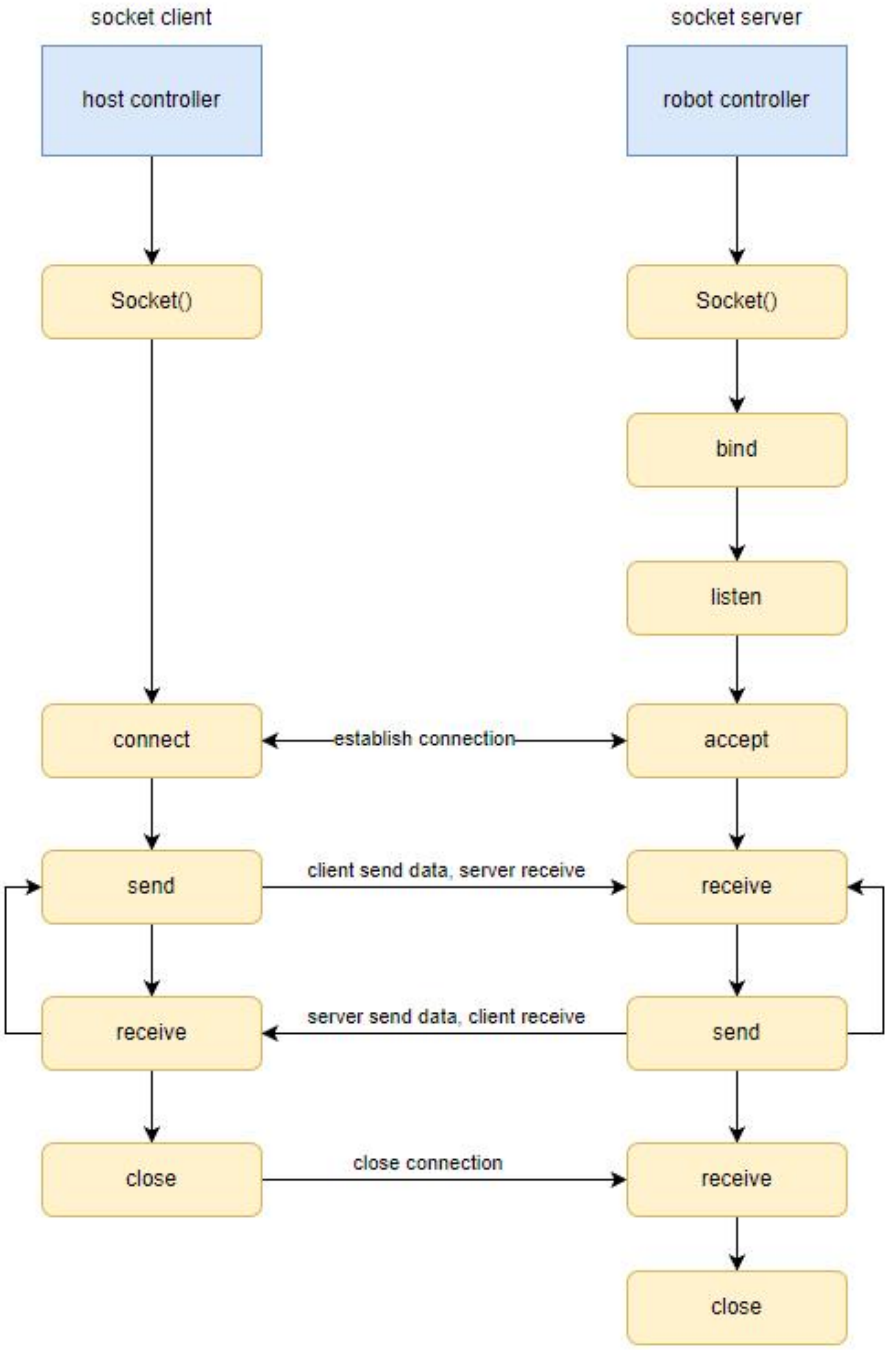


Fig. 1-1 Overview of Rob INTF based on TCP socket

2.1. Typical application scenarios

All rights reserved © Shanghai Bioyond Technology Co., Ltd

Based on TCP socket mentioned in Fig. 1-1, Rob INTF can support several typical application scenarios between host controller and robot controller. These scenarios can be divided into three categories:

1) The host controller requests the robot controller to perform some actions, see Fig. 2-2;

2) The host controller gets status information or other data from the robot controller, see Fig. 2-3;

3) The robot controller actively uploads status information or other data to the host controller, see Fig. 2-4.

The following activity sequence diagrams show the interaction between the host controller and the robot controller.

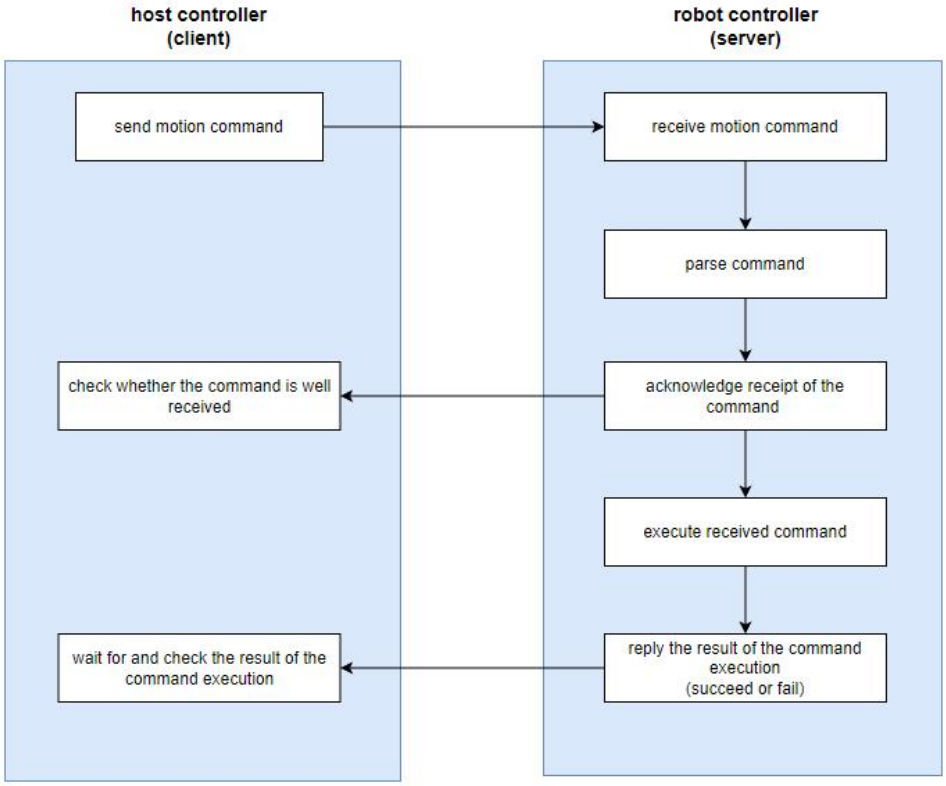


Fig. 2-2 Activity sequence diagram of scenario 1

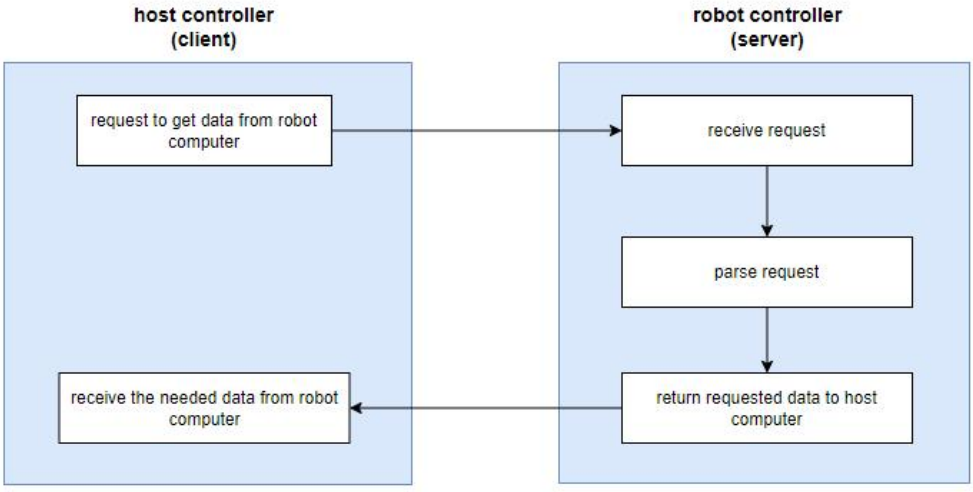


Fig. 2-3 Activity sequence diagram of scenario 2

All rights reserved © Shanghai Bioyond Technology Co., Ltd

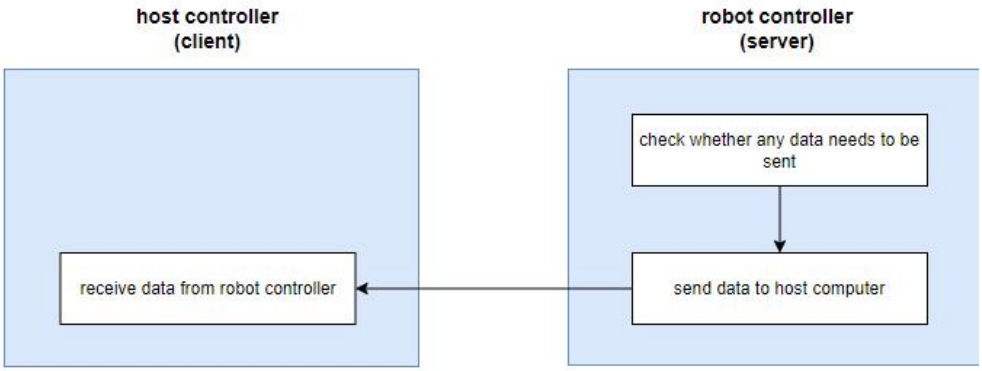


Fig. 2-4 Activity sequence diagram of scenario 3

2.2. IP address & Port Number

The host controller communicates with robot controller through the IP address and port number shown in Table 2-1.

Table 2-1 IP address and port number of Bioyond Rob INTF

IP Addr	Port	Description
192.168.1.220 (default)	56788	The IP address here does not have to be the value in this table. It depends on the actual user configuration (Bioyond robot controller IP address is configurable), but the port number is not configurable(should be fixed value 56788).

2.3. Interface definition

Rob INTF supports request and response data in JSON format. The host controller can send JSON data in an agreed format to request the robot controller to perform some instructions, and receive the correct format of JSON data feedback sent from the robot controller. One example of Rob INTF data is as follows. Tip: Send and receive data should both end with '\n'.

Command from host controller to robot controller:
{"BRCAPI":"1.0","cmd":"setSysVarI","params":[{"index":0,"value":1},{"index":1,"value":1}],"id":1}\n

Acknowledge receipt of the command:
Success:
{"BRCAPI":"1.0","ret":null,"id":1}\n
Fail:
{"BRCAPI":"1.0","error":{"code":10,"message":"error message"},"id":1}\n

Reply the result of the command execution:
{"BRCAPI":"1.0","reply":"CmdDone","id":1}\n

All rights reserved © Shanghai Bioyond Technology Co., Ltd

2.3.1. Request/command object

A request/command object from host controller contains the following members listed in Table 2-2.

Table 2-2 Description of request/command object

Object	Type	Description
BRCAPI	string	Specifying the protocol version, the current version is "1.0"
cmd	string	Name of command to be executed, see chapter 2.4 for the detailed list of supported commands.
params		Structured parameter values required by the calling method, this member parameter can be ignored if no need to send parameters for some commands.
id	int	Command id, which is used to correlate the request and response between the client and the server. When the server responds to the client's command, it needs to reply with the same id value to indicate which command is currently being processed. Therefore, the client can distinguish which command is being processed by setting a new id value for a new command, for example, increment the id by 1 for each new command.

2.3.2. Acknowledge/return object

For each motion command from host to robot controller to control robot, robot controller will send an acknowledge message to host to tell it the command is well received or not. In addition, for commands that intend to get data from robot controller, the robot controller will return the required data to the host. An acknowledge/return object from robot controller to host contains the following members listed in Table 2-3.

Table 2-3 Description of acknowledge/return object

Object	Type	Description
BRCAPI	string	Specifying the protocol version, the current version is "1.0"
ret		This field is only included if the robot controller successfully responds to the command from the host. Its value depends on the command from the host.
error		This field is only included if the robot controller fails to respond to the command from the host. It contains two sub-fields: error code and error message, which can tell the host controller what went wrong during the parsing of the command.
params		Structured parameter values required by the calling method, this member parameter can be ignored.
id	int	Command id, which is used to correlate the request and response between the client and the server. Note: id=-1 will be returned when JSON parsing error occurs.

All rights reserved © Shanghai Bioyond Technology Co., Ltd

2.3.3. Result reply object

In scenario 1 mentioned in chapter 2.1, robot controller will send command execution result to the host for some commands. This kind of result reply object contains the the following members listed in Table 2-4.

Table 2-4 Description of result reply object

Object	Type	Description
BRCAPI	string	Specifying the protocol version, the current version is "1.0"
reply		Reply the command execution result.
id	int	Command id, which is used to correlate the request and response between the client and the server.

2.4. Supported commands

The list of currently supported commands for communication between the robot controller and the host controller is shown in Table 2-5. Detailed descriptions about each commands, including the command format, can be found in Chapter 3.

Table 2-5 Supported commands list

Index	Command name	Description	Notes
1	setSysVarI	Set integer system variables.	
2	getSysVarI	Get integer system variables.	not need a result reply.
3	switchToAuto	Switch to auto mode.	
4	switchToManual	Switch to manual mode.	
5	remoteEnable	Enable the robot.	
6	remoteDisable	Disable the robot.	
7	remoteStart	Start/resume the robot program.	If the robot program is stopped, this command will execute the program from the start of main() function. If the robot program is paused, this command will resume running the program from the paused point of the program.
8	remotePause	Pause the robot program.	
9	remoteStop	Stop the robot program.	
10	remoteClearError	Clear errors.	
11	moveBackwards	Require the robot to move back to the starting point along the path it just traveled.	
12	stopMoveBackwards	Stop the robot moving backwards along the path.	
13	setSysParaI	Set integer system parameters.	

All rights reserved © Shanghai Bioyond Technology Co., Ltd

14	getSysParaI	Get integer system parameters.	
15	setSysParaF	Set floating point system parameters.	
16	getSysParaF	Get floating point system parameters.	
17	getCtrlMode	Get the control mode.	not need a result reply.
18	getEnableStatus	Get the enable status.	not need a result reply.
19	getProgState	Get the running status of the robot program.	not need a result reply.
20	getJointInfo	Get the current position of each joints of the robot.	not need a result reply.
21	getRobotErr	Get the robot error.	not need a result reply.
22	getLuaAppStatus	Get the status of the robot application. The supported status values include uninitialized and initialized.	not need a result reply.
23	getFrameWithU0T0	Get the current pose information of the robot, including position (X Y Z) and orientation (Rx Ry Rz). Note that the pose is calculated based on the default tool(T0) and user(U0) coordinate system.	not need a result reply.
24	getSysState	Get the overall system state of the robot controller.	not need a result reply.

2.5. Error Code

When the response fails, the error code and error message in the return message are described below:

Error code	Error message	Description
01	Invalid request	Invalid request, protocol version error.
02	Invalid command name	Invalid command. The command is not supported by robot controller yet.
03	Index exceeded allowed range	Index is out of range when setting or getting system variables.
04	Internal error	Internal error occurred on the robot controller.
05	Parse error	Received a non-compliant message that resulted in a parsing error
06	Operation failed, controller is in safety mode	The controller is in safety mode and this operation is not allowed.
07	Operation failed, internal error	Controller internal error.
08	Operation failed, can not move in	Controller is in disabled state and does

All rights reserved © Shanghai Bioyond Technology Co., Ltd

	disable state	not support robot motion command.
09	Operation failed, can not start/pause/stop in drag-go state	The controller is in drag-go state and these operations are not allowed.
10	Operation failed, Motion enable/disable must be in auto mode	The enable/disable operation must be performed in automatic mode.
11	Operation failed, controller is in Record-Play state	Controller is in the Record-Play state, not allowed to execute the move-backwards command.
12	Operation failed, lua program is running	Robot Lua program is running, the move-backwards command is not allowed.
13	Parse error: index type or value type is invalid.	
14	Operation failed, robot is moving now	The robot is moving, move-backwards command is not allowed.
15	Reserved	
16	Reserved	
17	Operation failed, can not stop moving backwards in other mode	The “stopMoveBackwards” command is not allowed if the robot is not in move-backwards state.
18	Received cmd buffer is full, check whether the package ends with '\n'	

All rights reserved © Shanghai Bioyond Technology Co., Ltd

3. Service

Under the premise of following the communication interface protocol specification, this robot controller can provide a series of services to the host controller. In this chapter, each of the services will be described in details to explain how to use it.

3.1. System variable/parameter service

System variables and system parameters have many similarities, so in most cases they can play the same role. However, since the processing of system variables involves response replies and execution result replies, therefore system variables are used to perform the execution that requires the robot to run one piece of program in Bioyond robot controller. The system parameters are only used as a representation of the data between the host and the robot controller, such as the coordinate value of the target point, speed, acceleration and other parameters.

3.1.1. Set integer system variable

This command supports setting multiple system variables at one time in the form of array. The example format of the request is shown as below. Note that arrays are enclosed in parentheses. If you want to set a single variable, set it as an array of length 1.

```
{"BRCAPI":"1.0","cmd":"setSysVarI","params":[{"index":0,"value":10},{"index":1,"value":20},{"index":2,"value":30}],"id":1}
```

Parameters definition in "params" field:

Parameter	Type	Range of Value	Description
index	int	0~ 19	System variable index
value	int	-2^{31}~ $2^{31}-1$	System variable value

There are two types of acknowledge from robot controller to host controller, depending on whether this command is parsed successfully, see follows as example.

```
Success:
{"BRCAPI":"1.0","ret":null,"id":1}\n
Fail:
{"BRCAPI":"1.0","error":{"code":10,"message":"error message"},"id":1}\n
```

Currently in Bioyond robot controller, system variables are used to perform the execution that requires the robot to run one piece of program. Therefore, each "setSysVarI" command from the host controller should expect a feedback from robot controller to indicate whether the robot successfully completed the program. This kind of feedback is named result reply, first introduced in chapter 2.3.3. For "setSysVarI" command, the result reply format is as follows. The index and value in the "reply" field are also system variables(system out variables) to indicate program execution result, etc.

```
{"BRCAPI":"1.0","reply":{"index":0,"value":1},"id":1}\n
```

All rights reserved © Shanghai Bioyond Technology Co., Ltd

3.1.2. Get integer system variable

This command supports getting multiple system variables at one time in the form of array. The format of the request can be seen as below. Note that arrays are enclosed in parentheses. If you want to get a single variable, get it as an array of length 1.

{"BRCAPI":"1.0","cmd":"getSysVarI","params":[{"index":0},{"index":1},{"index":2}],"id":1}\n

Parameter definition in "params" field:

Parameter	Type	Range of Value	Description
index	int	0 ~ 19	System variables address

For "getSysVarI" command, robot controller will return the required data to the host. There are two types of returns from robot controller to host controller, depending on whether this command is parsed successfully, see follows as example.

Success:

{"BRCAPI":"1.0","ret":[{"index":0,"value":10},{"index":1,"value":20},{"index":2,"value":30}],"id":1}\n

Fail:

{"BRCAPI":"1.0","error":{"code":10,"message":"error message"},"id":1}\n

3.1.3. Set integer system parameter

This command supports setting multiple integer system parameters at one time in the form of array. The example format of the request is shown as below. Note that arrays are enclosed in parentheses. If you want to set a single parameter, set it as an array of length 1.

{"BRCAPI":"1.0","cmd":"setSysParaI","params":[{"index":0,"value":10},{"index":1,"value":20},{"index":2,"value":30}],
"id":1}

Parameters definition in "params" field:

Parameter	Type	Range of Value	Description
index	int	0~ 49	System parameter index
value	int	-2^{31} ~ 2^{31}-1	System parameter value

The format of the returned data for this command is as below:

Success:

{"BRCAPI":"1.0","ret": null,"id":1}\n

Fail:

{"BRCAPI":"1.0","error":{"code":10,"message":"error message"},"id":1}\n

3.1.4. Get integer system parameter

This command supports getting multiple integer system parameters at one time in the form of array. The format of the request can be seen as below. Note that arrays are enclosed in parentheses. If you want to get a single parameter, get it as an array of length 1.

All rights reserved © Shanghai Bioyond Technology Co., Ltd

{"BRCAPI":"1.0","cmd":"getSysParaI","params":[{"index":0},{"index":1},{"index":2}],"id":1}\n

Parameter definition in "params" field:

Parameter	Type	Range of Value	Description
index	int	0 ~ 49	System parameter address

For "getSysParaI" command, robot controller will return the required data to the host. There are two types of returns from robot controller to host controller, depending on whether this command is parsed successfully, see follows as example.

Success:

{"BRCAPI":"1.0","ret":[{"index":0,"value":10},{"index":1,"value":20},{"index":2,"value":30}],"id":1}\n

Fail:

{"BRCAPI":"1.0","error":{"code":10,"message":"error message"},"id":1}\n

3.1.5. Set floating point system parameter

This command supports setting multiple float system parameters at one time in the form of array. The example format of the request is shown as below. Note that arrays are enclosed in parentheses. If you want to set a single parameter, set it as an array of length 1.

{"BRCAPI":"1.0","cmd":"setSysParaF","params":[{"index":0,"value":1.23},{"index":1,"value":4.56},{"index":2,"value":7.89}],"id":1}

Parameters definition in "params" field:

Parameter	Type	Range of Value	Description
index	int	0~ 49	System parameter index
value	float	Range of floating type	System parameter value

The format of the returned data for this command is as below:

Success:

{"BRCAPI":"1.0","ret": null,"id":1}\n

Fail:

{"BRCAPI":"1.0","error":{"code":10,"message":"error message"},"id":1}\n

3.1.6. Get floating point system parameter

This command supports getting multiple float system parameters at one time in the form of array. The format of the request can be seen as below. Note that arrays are enclosed in parentheses. If you want to get a single parameter, get it as an array of length 1.

{"BRCAPI":"1.0","cmd":"getSysParaF","params":[{"index":0},{"index":1},{"index":2}],"id":1}\n

All rights reserved © Shanghai Bioyond Technology Co., Ltd

Parameter definition in "params" field:

Parameter	Type	Range of Value	Description
index	int	0 ~ 49	System parameter address

For "getSysParaF" command, robot controller will return the required data to the host. There are two types of returns from robot controller to host controller, depending on whether this command is parsed successfully, see follows as example.

Success:
{"BRCAPI":"1.0","ret":[{"index":0,"value":1.23},{"index":1,"value":4.56},{"index":2,"value":7.89}],"id":1}\n
Fail:
{"BRCAPI":"1.0","error":{"code":10,"message":"error message"},"id":1}\n

3.2. Control service

3.2.1. Switch to auto control mode

This command switches the robot control mode to "auto" mode. In "auto" mode, jogging robot is not allowed and the robot speed is higher than that of the manual mode. The format of the request can be seen as below. Note: "params" field is not needed for this command.

{"BRCAPI":"1.0","cmd":"switchToAuto","id":1}\n

There are two types of acknowledge from robot controller to host controller, depending on whether this command is parsed successfully, see follows as example.

Success:
{"BRCAPI":"1.0","ret":null,"id":1}\n
Fail:
{"BRCAPI":"1.0","error":{"code":10,"message":"error message"},"id":1}\n

Result reply will be sent to host controller to indicate whether the command is executed successfully or not. The result reply format for this command is as follows.

Execution success:
{"BRCAPI":"1.0","reply":"CmdDone","id":1}\n
Execution timeout or failed:
{"BRCAPI":"1.0","reply":"CmdTimeout","id":1}\n

3.2.2. Switch to manual control mode

This command switches the robot control mode to "manual" mode. In "manual" mode, user can jog robot and robot speed is lower than that of the auto mode. The format of the request can be seen as below. Note: "params" field is not needed for this command.

{"BRCAPI":"1.0","cmd":"switchToManual","id":1}\n

All rights reserved © Shanghai Bioyond Technology Co., Ltd

The format of command acknowledge and result reply for this command is the same as the one in chapter 3.2.1. No more details here.

3.2.3. Enable robot

This command enables robot so that robot can move as desired. The format of the request can be seen as below. Note: "params" field is not needed for this command.

```
{"BRCAPI":"1.0","cmd":"remoteEnable","id":1}\n
```

The format of command acknowledge and result reply for this command is the same as the one in chapter 3.2.1. No more details here.

3.2.4. Disable robot

This command disables robot so that robot can not move and the user can approach it very safely. The format of the request can be seen as below. Note: "params" field is not needed for this command.

```
{"BRCAPI":"1.0","cmd":"remoteDisable","id":1}\n
```

The format of command acknowledge and result reply for this command is the same as the one in chapter 3.2.1. No more details here.

3.2.5. Start running robot program

This command starts or continues the execution of the robot program. If the robot program is stopped, this command will execute the program from the start of main() function. If the robot program is paused, this command will resume running the program from the paused point of the program. The format of the request can be seen as below. Note: "params" field is not needed for this command.

```
{"BRCAPI":"1.0","cmd":"remoteStart","id":1}\n
```

The format of command acknowledge and result reply for this command is the same as the one in chapter 3.2.1. No more details here.

3.2.6. Pause running robot program

This command pauses the execution of the robot program. The robot program pointer pauses at the currently executing program statement. If "remoteStart" is received now, the robot will continue to execute the program from this point. The format of the request can be seen as below. Note: "params" field is not needed for this command.

```
{"BRCAPI":"1.0","cmd":"remotePause","id":1}\n
```

The format of command acknowledge and result reply for this command is the same as the one in chapter 3.2.1. No more details here.

All rights reserved © Shanghai Bioyond Technology Co., Ltd

3.2.7. Stop running robot program

This command stops the execution of the robot program. The robot will abandon all the program statements that have not yet been executed when this command is received. Then the program pointer will go back to the start of the main() function. If "remoteStart" is received now, the robot will start to re-execute the program from the start point. The format of the request can be seen as below. Note: "params" field is not needed for this command.

```
{"BRCAPI":"1.0","cmd":"remoteStop","id":1}\n
```

The format of command acknowledge and result reply for this command is the same as the one in chapter 3.2.1. No more details here.

3.2.8. Clear error

This command clears robot errors if any. If there are no errors but the request is received, the robot controller will still reply to host controller that the command was successfully executed. The format of the request can be seen as below. Note: "params" field is not needed for this command.

```
{"BRCAPI":"1.0","cmd":"remoteClearError","id":1}\n
```

The format of command acknowledge and result reply for this command is the same as the one in chapter 3.2.1. No more details here.

3.2.9. Move robot backwards

This command requires the robot to move back to the starting point along the path it just traveled. A typical application of this command is the automatic recovery from fault. In the event of a collision or other failure of the robot, this command allows the robot to safely move from the failure site back to the starting point--a relatively safe point preset by the user. The format of the request can be seen as below. Note: "params" field is not needed for this command.

```
{"BRCAPI":"1.0","cmd":"moveBackwards","id":1}\n
```

The format of command acknowledge for this command is the same as the one in chapter 3.2.1.
Result reply will be sent to host controller to indicate whether the command is executed successfully or not. The result reply format for this command is as follows. "CmdTimeout" does not apply to this command because the robot's "move-backwards" time can be very long and unpredictable, depending on the path of the "move-backwards" and the speed set during the "move-backwards".

Execution success:

```
{"BRCAPI":"1.0","reply":"CmdDone","id":1}\n
```

Execution failed:

```
{"BRCAPI":"1.0","reply":"Cmd Execution error","id":1}\n
```

3.2.10. Stop moving backwards

This command stops the robot's moving-backwards process. It allows the user to terminate the execution of "move backwards" action at any time according to the actual situation. The format of the request can be seen as below. Note: "params" field is not needed for this command.

All rights reserved © Shanghai Bioyond Technology Co., Ltd

```
{"BRCAPI":"1.0","cmd":"stopMoveBackwards","id":1}\n
```

The format of command acknowledge and result reply for this command is the same as the one in chapter 3.2.1. No more details here.

3.3. Query service

Bioyond robot provides a series of query services to allow host controller to obtain the robot status and other data. All query services feed back data in real time, so there is no need to execution command to operate the robot. Therefore, for query services, acknowledge of command and returned data is combined into a single feedback to host controller.

3.3.1. Get robot control mode

This command gets the robot control mode. The format of the request can be seen as below. Note: "params" field is not needed for this command.

```
{"BRCAPI":"1.0","cmd":"getCtrlMode","id":1}\n
```

The format of the returned data for this command is as below:

```
Success:
{"BRCAPI":"1.0","ret":{"value":1},"id":1}\n
Fail:
{"BRCAPI":"1.0","error":{"code":10,"message":"error message"},"id":1}\n
```

Parameters definition of the returned data for this command is listed in below table.

Parameter	Type	Range of Value	Description
value	int	0, 1	Current robot control mode. 0: manual mode 1: auto mode Other values are illegal.

3.3.2. Get robot enable status

This command gets the status indicating whether robot is enabled. The format of the request can be seen as below. Note: "params" field is not needed for this command.

```
{"BRCAPI":"1.0","cmd":"getEnableStatus","id":1}\n
```

The format of the returned data for this command is as below:

```
Success:
{"BRCAPI":"1.0","ret":{"value":1},"id":1}\n
Fail:
{"BRCAPI":"1.0","error":{"code":10,"message":"error message"},"id":1}\n
```

Parameters definition of the returned data for this command is listed in below table.

All rights reserved © Shanghai Bioyond Technology Co., Ltd

Parameter	Type	Range of Value	Description
value	int	0, 1	Current enable status of the robot. 0: disabled 1: enabled Other values are illegal.

3.3.3. Get robot program state

This command gets the state of robot program. The format of the request can be seen as below. Note: "params" field is not needed for this command.

```
{"BRCAPI":"1.0","cmd":"getProgState","id":1}\n
```

The format of the returned data for this command is as below:

```
Success:
{"BRCAPI":"1.0","ret":{"value":1},"id":1}\n
Fail:
{"BRCAPI":"1.0","error":{"code":10,"message":"error message"},"id":1}\n
```

Parameters definition of the returned data for this command is listed in below table.

Parameter	Type	Range of Value	Description
value	int	0, 1, 2, 3	Current robot program running status: 0: Program stopped 1: Program running 2: Program paused 3: Program error Other values are illegal.

3.3.4. Get robot joint information

This command supports to get joint information of the robot. Currently only joint position will be sent back to host. This command can be expanded to get more joint information than just the joint position. The format of the request can be seen as below. Note: "params" field is not needed for this command.

```
{"BRCAPI":"1.0","cmd":"getJointInfo","id":1}\n
```

The format of the returned data for this command is as below. In the following example, robot controller returns 2 robot joint position values and 1 external joint position values. Actually the number of robot joints and the number of external axes can be much more than that in the following example. For example, for a 6-axes robot with 2 external axes, the returned data contains 6 joints values and 2 external joints values.

```
Success:
{"BRCAPI":"1.0","ret":{"joints":[{"index":0,"value":1.1},{"index":1,"value":2.2}],"extjoint":[{"index":0,"value":3.3}]},"id
":1}\n
Fail:
{"BRCAPI":"1.0","error":{"code":10,"message":"error message"},"id":1}\n
```

All rights reserved © Shanghai Bioyond Technology Co., Ltd

Parameters definition of the returned data for this command is listed in below table.

Parameter	Type	Range of Value	Description
index	int	0~7	Maximum supported robot joints number and external axes number. Bioyond robot controller supports maximum 8 robot joints and 8 external axes.
value	double		

3.3.5. Get robot error

This command gets the robot error from robot controller. The format of the request can be seen as below. Note: "params" field is not needed for this command.

```
{"BRCAPI":"1.0","cmd":"getRobotErr","id":1}\n
```

The format of the returned data for this command is as below. If no error in robot controller, the value of "errCode" and "errMsg" field are both null. If there are errors in robot controller, "errCode" field contains the error code, and the "errMsg" field contains the corresponding error message. Currently, robot controller returns up to 10 errors at a time.

Success and there is no error in robot controller:

```
{"BRCAPI":"1.0","ret":[{"errCode":null,"errMsg":null}],"id":1}\n
```

Success and there are errors in robot controller:

```
{"BRCAPI":"1.0","ret":[{"errCode":error code,"errMsg":"xxx"},{"errCode":error code,"errMsg":"xxx"}],"id":1}\n
```

Fail:

```
{"BRCAPI":"1.0","error":{"code":10,"message":"error message"},"id":1}\n
```

Parameters definition of the returned data for this command is listed in below table.

Parameter	Type	Range of Value	Description
errCode	int		Error code is a decimal integer.
errMsg	string		errMsg is string type and describes the error information including possible cause, consequences and actions that should be taken.

3.3.6. Get Lua application execution status

This command gets the robot application execution status. As mentioned above in 3.3.1, in general, system variables are used to make process logic judgments for robot programs. The acquisition and resetting of system variables are usually implemented in Lua programs. Therefore, we need to ensure that the Lua application program is already running before handling system variables, which is an appropriate way. This flag can usually be set after the initialization part of a Lua application(including the initialization/resetting of system variables) is complete and before the main loop begins. The host can get this status to determine whether the robot controller is ready to receive system variables for program logic processing.

The format of the request can be seen as below. Note: "params" field is not needed for this command.

```
{"BRCAPI":"1.0","cmd":"getLuaAppStatus","id":1}\n
```

All rights reserved © Shanghai Bioyond Technology Co., Ltd

The format of the returned data for this command is as below:

Success:
{"BRCAPI":"1.0","ret":{"value":1},"id":1}\n
Fail:
{"BRCAPI":"1.0","error":{"code":10,"message":"error message"},"id":1}\n

Parameters definition of the returned data for this command is listed in below table.

Parameter	Type	Range of Value	Description
value	int	0, 1	Current robot program running status: 0: uninitialized 1: initialized Other values are illegal.

3.3.7. Get robot pose information

This command gets the current pose information of the robot, including position (X Y Z) and orientation (Rx Ry Rz). Note that the pose is calculated based on the default tool(T0) and user(U0) coordinate system. The format of the request can be seen as below. Note: "params" field is not needed for this command.

{"BRCAPI":"1.0","cmd":"getFrameWithU0T0","id":1}\n

The format of the returned data for this command is as below.

Success:
{"BRCAPI":"1.0","ret":{"X":1.1, "Y":2.2, "Z":3.3, "Rx":4.4, "Ry":5.5, "Rz":6.6},"id":1}\n
Fail:
{"BRCAPI":"1.0","error":{"code":1,"message":"error message"},"id":1}\n

3.3.8. Get system state

This command gets the overall system state of the robot controller. In fact, the controller startup process will take a certain amount of time. For example, the communication network between the controller and the servo drive takes tens of seconds to set up, which is defined as "init" state. However, communication between the controller and the host may have been established prior to this. In this case, some instructions sent by the host to the controller may not be responded normally. This command can be used by the host to check whether the controller has been successfully initialized. In addition, this command can also provide more information about the status of the controller to the host.

The format of the request can be seen as below.

{"BRCAPI":"1.0","cmd":"getSysState","id":1}\n

The format of the returned data for this command is as below.

Success:
{"BRCAPI":"1.0","ret":{"value":0},"id":1}\n
Fail:

All rights reserved © Shanghai Bioyond Technology Co., Ltd

{"BRCAPI":"1.0","error":{"code":1,"message":"error message"},"id":1}\n

Parameters definition of the returned data for this command is listed in below table.

Parameter	Type	Range of Value	Description
value	int	0-5, integer	Current overall state of the controller. 0: init 1: idle 2: running 3: error 4: paused 5: enabled Other values are illegal.

4. Reference example

In this chapter, a few examples will be provided to show how host controller interacts with Bioyond robot controller through Rob INTF to control the robot, such as query, control and other operations. The examples are shown in the form of a diagram, which can clearly show the entire interaction process.

4.1. Host controls robot to run specific program

A typical use case is that the host controller tries to control the robot to run specific program. Fig. 4-1 shows the entire interaction between host controller and robot controller to make robot run specific programs. Developers of host controller can refer to this diagram for this use case.

All rights reserved © Shanghai Bioyond Technology Co., Ltd

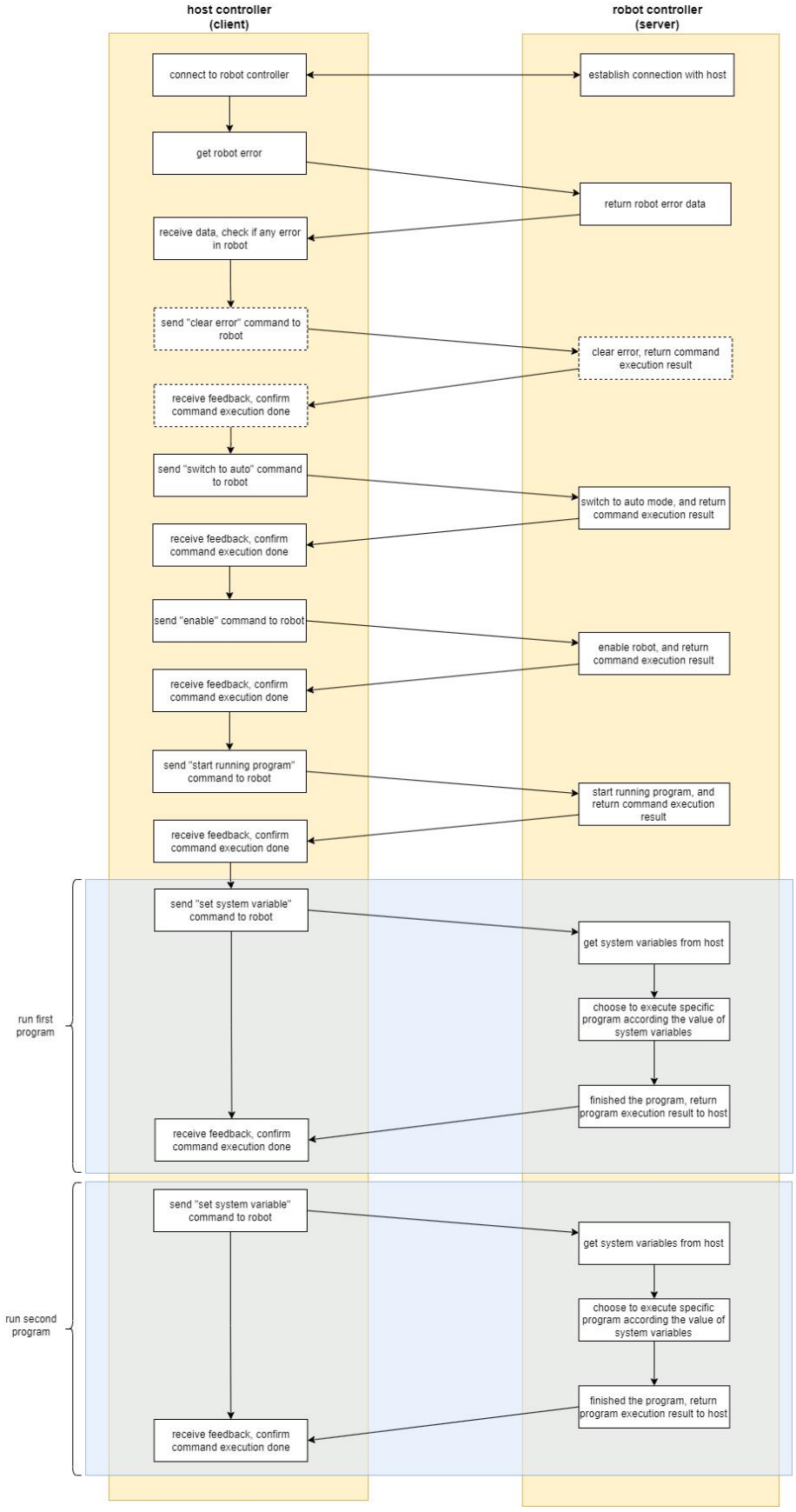


Fig. 4-1 Reference example: host controls robot to run specific program

All rights reserved © Shanghai Bioyond Technology Co., Ltd

4.2. Host controls robot to recover from error

Some faults may occur when robot is working, such as collision with personnel. Bioyond robot can recover from fault site almost without need of manual intervention. Fig. 4-2 gives an example for this use case.

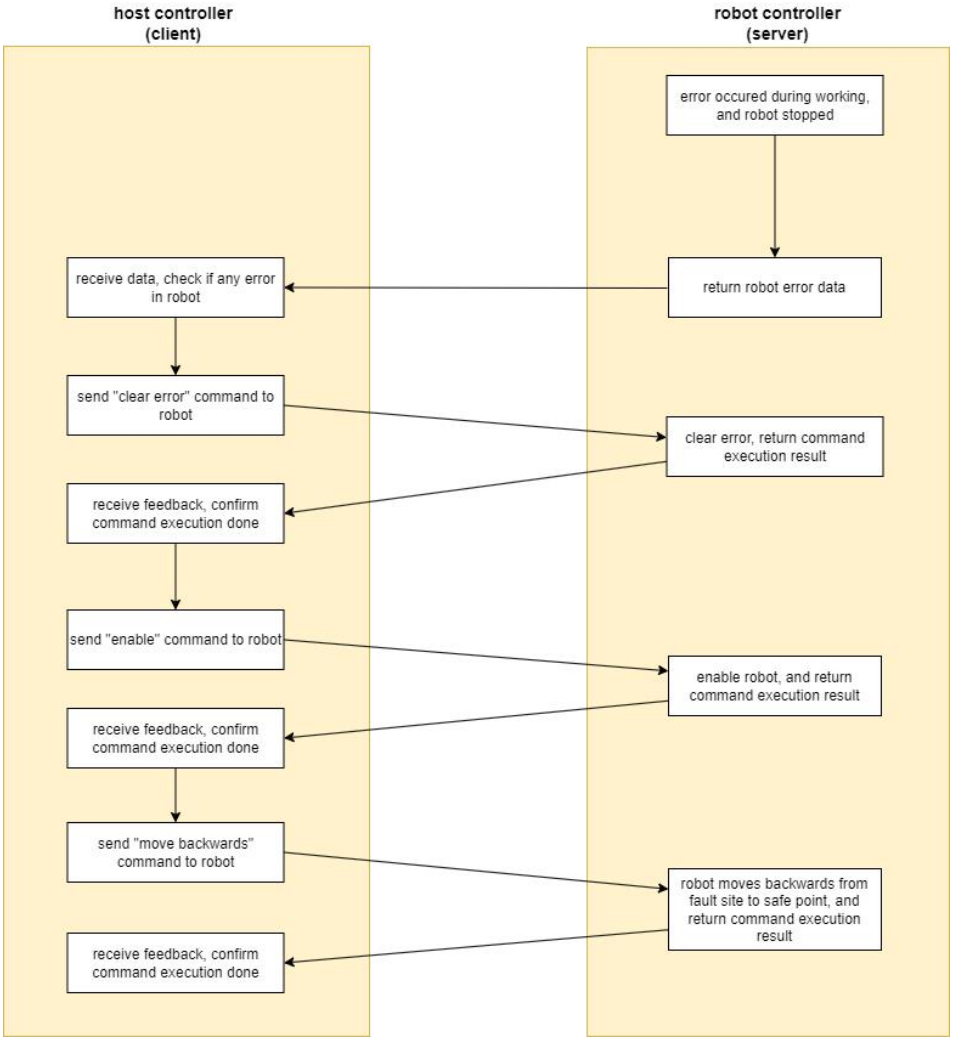


Fig. 4-2 Reference example: host controls robot to recover from error

All rights reserved © Shanghai Bioyond Technology Co., Ltd